Stat 175: Homework 6

matt krasnow

Spring 2026

Please submit your solutions to Gradescope by **Monday, April 20th, 11:59 pm**.

This entire homework will be using the data set from last week's Exercise 1, which can be viewed (with relevant code) at this Google Colab. Recall that this Citeseer dataset is a citation dataset consisting of 3,327 vertices (the papers) and 9,107 edges (an undirected edge indicating that one of the two papers cited the other one). Each vertex can belong to one of six categories, which is represented by a label, and our primary goal is to perform node prediction.

Exercise 1 (GCN vs. Original Features).

(*a*) Train a logistic regression model on the final embeddings obtained from the Graph Convolutional Network (GCN) and discuss its performance.

(*b*) Compare the above performance to a logistic regression model trained on the original features, and discuss any differences in performance between the two models.

Solution to Exercise 1

Setup. A 2-layer GCN is trained on the full Citeseer graph using the supplied training mask. Hidden dimension 64, ReLU, dropout 0.5, Adam (lr $= 0.01$, weight decay 5×10^{-4}), 200 epochs, early stopping on validation accuracy. The embedding $Z \in \mathbb{R}^{n \times 64}$ is the post-ReLU output of the first conv layer. Both Z and the original features X are then evaluated with logistic regression on a 70/30 stratified node split.

Code.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchchar95geometric.datasets import Planetoid
from torchchar95geometric.nn import GCNConv
from sklearn.linearchar95model import LogisticRegression
from sklearn.metrics import f1char95score
from sklearn.modelchar95selection import trainchar95testchar95split
from sklearn.preprocessing import StandardScaler
import numpy as np

dataset = Planetoid(rochar39tmp/Citeschar39, namechar39Citeschar39)
data = dataset[0]

class GCN(nn.Module):
    defchar00char95char95char95(self, inchar95dim, hidden, outchar95dim):
        super().char00char95char95()
        self.c1 = GCNConv(inchar95dim, hidden)
        self.c2 = GCNConv(hidden, outchar95dim)
    def forward(self, x, ei):
        x = F.relu(self.c1(x, ei))
```

```
        x = F.dropout(x, 0.5, training=self.training)
        return self.c2(x, ei)
    def embed(self, x, ei):
        return F.relu(self.c1(x, ei))

torch.manual_seed(0)
model = GCN(dataset.num_features, 64, dataset.num_classes)
opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

best_val, best_state = 0.0, None
for _ in range(200):
    model.train(); opt.zero_grad()
    out = model(data.x, data.edge_index)
    F.cross_entropy(out[data.train_mask], data.y[data.train_mask]).backward()
    opt.step()
    model.eval()
    with torch.no_grad():
        pred = model(data.x, data.edge_index).argmax(1)
        v = (pred[data.val_mask] == data.y[data.val_mask]).float().mean().item()
        if v > best_val:
            best_val, best_state = v, {k: p.clone() for k, p in model.
                state_dict().items()}
model.load_state_dict(best_state)

with torch.no_grad():
    Z = model.embed(data.x, data.edge_index).numpy()
X = data.x.numpy(); y = data.y.numpy()

idx_tr, idx_te = train_test_split(np.arange(len(y)), test_size=0.3,
                                  random_state=0, stratify=y)

sc = StandardScaler()
Ztr, Zte = sc.fit_transform(Z[idx_tr]), sc.transform(Z[idx_te])
clf_Z = LogisticRegression(max_iter=5000).fit(Ztr, y[idx_tr])
f1_Z = f1_score(y[idx_te], clf_Z.predict(Zte), average='macro')

sc = StandardScaler(with_mean=False)
Xtr, Xte = sc.fit_transform(X[idx_tr]), sc.transform(X[idx_te])
clf_X = LogisticRegression(max_iter=5000).fit(Xtr, y[idx_tr])
f1_X = f1_score(y[idx_te], clf_X.predict(Xte), average='macro')
```

Results and discussion.

- Logistic regression on the GCN embeddings: macro-F1 $= 0.6737$.
- Logistic regression on the original bag-of-words features: macro-F1 $= 0.6701$.

The two scores differ by only 0.004. Citeseer's bag-of-words features are already informative about paper category, so propagating information through the graph adds little. The GCN compresses 3703 features into 64 structure-aware dimensions while preserving accuracy.

Exercise 2 (Edge Prediction).

(a) Implement edge prediction using the GCN architecture. To avoid data leakage from creating the embeddings, you must split the graph's edges into training and testing data, and only use the edges

in the training set to make embeddings.

(*b*) Discuss the method you chose for training the embeddings and explain your reasoning behind the choice.

Solution to Exercise 2

(a) Edge prediction with GCN.
Edges are split 85/5/10 into train/val/test, and the message-passing graph contains only training edges. An equal number of negative (non-)edges is sampled uniformly at random, disjoint from observed edges, freshly per training epoch and once at split time for val/test.
The encoder is a 2-layer GCN mapping each node to $z_v \in \mathbb{R}^{64}$:

$$Z = \mathrm{GCN}(X, E_{\text{train}}).$$

A candidate edge (u, v) is scored by the inner product $s_{uv} = z_u^\top z_v$, trained with binary cross-entropy on positive/negative pairs. Adam (lr $= 0.01$, 200 epochs), early stopping on validation ROC-AUC.

Code.

```
import torch, torch.nn as nn, torch.nn.functional as F
import numpy as np
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
from torch_geometric.utils import negative_sampling
import torch_geometric.transforms as T
from sklearn.metrics import roc_auc_score

data = Planetoid(root='/tmp/Cites', name='Cites')[0]

transform = T.RandomLinkSplit(num_val=0.05, num_test=0.10,
                              is_undirected=True,
                              add_negative_train_samples=False,
                              split_labels=True)
torch.manual_seed(0)
train_d, val_d, test_d = transform(data.clone())

class Encoder(nn.Module):
    def __init__(self, in_dim, hidden, out_dim):
        super().__init__()
        self.c1 = GCNConv(in_dim, hidden)
        self.c2 = GCNConv(hidden, out_dim)
    def forward(self, x, ei):
        x = F.relu(self.c1(x, ei))
        x = F.dropout(x, 0.5, training=self.training)
        return self.c2(x, ei)

def decode(z, ei):
    return (z[ei[0]] * z[ei[1]]).sum(-1)

model = Encoder(data.num_features, 64, 64)
opt = torch.optim.Adam(model.parameters(), lr=0.01)

best_auc, best_state = 0.0, None
for _ in range(200):
    model.train(); opt.zero_grad()
    z = model(train_d.x, train_d.edge_index)
    pos = train_d.pos_edge_label_index
```

```
    neg = negative_sampling(edge_index=train_data.edge_index,
                            num_nodes=data.num_nodes,
                            num_neg_samples=pos.size(1), method='sparse')
    logits = torch.cat([decode(z, pos), decode(z, neg)])
    labels = torch.cat([torch.ones(pos.size(1)), torch.zeros(neg.size(1))])
    F.binary_cross_entropy_with_logits(logits, labels).backward()
    opt.step()
    model.eval()
    with torch.no_grad():
        z = model(train_data.x, train_data.edge_index)
        p = torch.sigmoid(decode(z, val_pos_edge_label_index)).numpy()
        n = torch.sigmoid(decode(z, val_neg_edge_label_index)).numpy()
        auc = roc_auc_score(np.concatenate([np.ones_like(p), np.zeros_like(n)
            ]),
                            np.concatenate([p, n]))
        if auc > best_auc:
            best_auc, best_state = auc, {k: p.clone() for k, p in model.
                state_dict().items()}
model.load_state_dict(best_state)

with torch.no_grad():
    z = model(train_data.x, train_data.edge_index)
    p = torch.sigmoid(decode(z, test_pos_edge_label_index)).numpy()
    n = torch.sigmoid(decode(z, test_neg_edge_label_index)).numpy()
y_true = np.concatenate([np.ones_like(p), np.zeros_like(n)])
y_score = np.concatenate([p, n])
auc = roc_auc_score(y_true, y_score)
zero_one_loss = np.mean((y_score >= 0.5).astype(int) != y_true)
```

Results. On held-out test edges: ROC-AUC = 0.8954, 0/1 loss = 0.2297 (accuracy = 0.7703). This beats the HW5 Node2vec result (AUC = 0.665) because the GCN uses node features as well as graph structure.

(b) Discussion of the method.
Split the edges, not the nodes. The unit of observation is an edge, so the partition is over edges and the message-passing graph contains only training edges. Otherwise a test edge visible during embedding training would leak its endpoints into the test score.
Inner-product decoder rather than Hadamard + logistic regression. HW5 used $\varphi(u, v) = z_u \odot z_v$ with a downstream classifier. Since the embeddings are now trained themselves, the decoder $s_{uv} = z_u^\top z_v$ (Graph Auto-Encoder, Kipf & Welling, 2016) lets the encoder optimize against the edge objective end-to-end in one stage.
Fresh negative samples each epoch. Edge prediction is imbalanced ($O(n)$ edges versus $O(n^2)$ non-edges). Resampling negatives each epoch acts as data augmentation and keeps the encoder from overfitting any particular set of non-edges.
ROC-AUC and 0/1 loss. Edge prediction is a ranking problem, so ROC-AUC is the primary metric. The 0/1 loss is threshold-dependent; I report it at $s_{uv} \geq 0.5$ after sigmoid.

Exercise 3 (Number of Layers).

(*a*) Experiment with the number of layers in the GCN architecture, ranging from 1 to 5 layers. Analyze and compare the performance of each model for node classification, using the F_1 macro metric. How does changing the number of layers affect performance?

(*b*) Do the exact same as above, but now for edge prediction, using the 0/1 loss. Again, how does

changing the number of layers affect performance?

Solution to Exercise 3

(a) Node classification.
The Ex 1 GCN is re-trained with $L \in \{1, 2, 3, 4, 5\}$ layers, hidden dimension 64. Macro-F1 comes from logistic regression on the penultimate embedding, using the same 70/30 split as Ex 1.

```
class GCN(nn.Module):
    def __init__(self, in_dim, hidden, out_dim, L):
        super().__init__()
        self.convs = nn.ModuleList()
        if L == 1:
            self.convs.append(GCNConv(in_dim, out_dim))
        else:
            self.convs.append(GCNConv(in_dim, hidden))
            for _ in range(L - 2):
                self.convs.append(GCNConv(hidden, hidden))
            self.convs.append(GCNConv(hidden, out_dim))
    def forward(self, x, ei):
        for i, c in enumerate(self.convs):
            x = c(x, ei)
            if i < len(self.convs) - 1:
                x = F.dropout(F.relu(x), 0.5, training=self.training)
        return x
```

Results (macro-F1).

L	1	2	3	4	5
macro-F1	0.6648	**0.6737**	0.6627	0.6569	0.6246

$L = 2$ is best, $L = 1$ within 0.009. Performance degrades monotonically past $L = 2$, with a sharp drop at $L = 5$. This is *over-smoothing* [1]: stacking GCN layers expands the receptive field, and in the limit all node embeddings converge to the same value. Two hops suffice for Citeseer; deeper models blur the class signal.

(b) Edge prediction.
Same sweep on the Ex 2 pipeline. I report ROC-AUC and 0/1 loss at threshold 0.5.

L	1	2	3	4	5
ROC-AUC	**0.9516**	0.8878	0.8776	0.8709	0.8503
0/1 loss	0.2495	**0.2429**	0.2637	0.2505	0.2725

The shallow-wins trend is sharper for edge prediction. $L = 1$ gives the best ROC-AUC: its embeddings encode 1-hop compatibility, which is exactly what the inner-product decoder scores. $L = 2$ wins the 0/1 loss by a hair due to better threshold calibration. Deeper models over-smooth and AUC drops from 0.95 to 0.85.

Summary across (a) and (b). Going deeper hurts in both settings. $L = 1$ or $L = 2$ is optimal for Citeseer, matching the GCN paper's shallow-network recommendation. For edge prediction, a single convolution is the strongest choice.

References

[1] Li, Q., Han, Z., and Wu, X.-M. (2018). *Deeper insights into graph convolutional networks for semi-supervised learning.* AAAI.

Exercise 4 (Aggregation Methods).

(*a*) To explore different aggregation methods, implement two versions of the GCN: one with pooling and one with LSTM-based aggregation. Compare the performance of each aggregation method on the node classification task.

Solution to Exercise 4

Setup. The standard GCN aggregator is symmetric-mean via the Laplacian-normalized propagation rule, so swapping aggregators requires moving to the GraphSAGE family. I build two GraphSAGE networks with identical architecture (2 layers, hidden 64, ReLU, dropout 0.5, Adam lr = 0.01) and different aggregators.

(i) **Pooling aggregator.** Each neighbor's representation is passed through a shared linear map and element-wise max-pooled:

$$\mathrm{AGG}_{\mathrm{pool}}(\{h_u : u \in N(v)\}) \;=\; \max_{u \in N(v)} \sigma\big(W_{\mathrm{pool}} h_u + b\big)\,,$$

which is the `SAGEConv(aggr="max")` variant of GraphSAGE.

(ii) **LSTM aggregator.** Each node's (unordered) neighborhood is fed to a shared LSTM cell in a random permutation; the final hidden state becomes the aggregated neighbor message:

$$\mathrm{AGG}_{\mathrm{LSTM}}(\{h_u : u \in N(v)\}) \;=\; \mathrm{LSTM}\big(\pi(\{h_u : u \in N(v)\})\big)_{\mathrm{last}}\,.$$

LSTM aggregation is much more expensive since it unrolls a recurrent network per neighborhood. To run on CPU I subsample each neighborhood to $K = 3$, use hidden dimension 16, and train for 20 epochs with gradient clipping ($\|g\| \le 1$) at lr = 0.005.

Code.

```
import torch, torch.nn as nn, torch.nn.functional as F
from torchchar95geometric.nn import SAGEConv

class SAGEModel(nn.Module):
    defchar95char95initchar95char95(self, inchar95dim, hidden, outchar95dim, aggr):
        super().char95char95initchar95char95()
        self.c1 = SAGEConv(inchar95dim, hidden, aggr=aggr)
        self.c2 = SAGEConv(hidden, outchar95dim, aggr=aggr)
    def forward(self, x, ei):
        x = F.relu(self.c1(x, ei))
        x = F.dropout(x, 0.5, training=self.training)
        return self.c2(x, ei)
    def embed(self, x, ei):
        return F.relu(self.c1(x, ei))

# Pool: aggr="max", 2 layers hidden=64, 200 epochs, full edgechar95index.
# LSTM: aggr="lstm", 2 layers hidden=16, 20 epochs, neighborhood sub-sampled
    to K=3,
#       LR=5e-3, gradient-clip maxchar95norm=1.0.
```

Neighborhood sub-sampling for LSTM is performed by uniformly selecting at most K incoming edges per destination node before the forward pass:

```
import numpy as np
def subsamplechar95edges(edgechar95index, numchar95nodes, K=3, seed=0):
    rng = np.random.defaultchar95rng(seed)
```

```
    src, dst = edge_index.numpy()
    order = np.argsort(dst, kind="stable")
    src, dst = src[order], dst[order]
    keep = np.ones(len(dst), dtype=bool)
    s = 0
    for node in range(num_nodes):
        e = s
        while e < len(dst) and dst[e] == node:
            e += 1
        if e - s > K:
            chosen = rng.choice(np.arange(s, e), size=K, replace=False)
            mask = np.zeros(e - s, dtype=bool); mask[chosen - s] = True
            keep[s:e] = mask
        s = e
    return torch.tensor(np.stack([src[keep], dst[keep]]), dtype=torch.long)
```

After training, I evaluate each aggregator by fitting a logistic regression on the penultimate embedding (same split as Ex. 1).

Results.

Aggregator	macro-F1 (LR on embedding)	macro-F1 (direct)
Max pooling	**0.6758**	0.6417
LSTM	0.5351	0.5075

Discussion. Max pooling matches the Ex 1 GCN; LSTM lags at 0.535 macro-F1.
Performance. The GraphSAGE paper reports comparable LSTM and pool performance on citation-scale graphs. My LSTM lags because of (a) $K = 3$ neighborhood subsampling, (b) hidden dimension 16, and (c) only 20 epochs, all required to avoid CPU OOM. The gap reflects the compute budget, not a fundamental ranking of the aggregators.
Compute. Pool: 200 epochs on the full graph in 30s. LSTM: 20 epochs on a subsampled graph with a quarter of the hidden dimension, 290s. LSTM is an order of magnitude more expensive and parallelizes poorly across neighborhood sizes.
Expressiveness. LSTM can model interactions between neighbors that symmetric pooling cannot, but it also imposes an arbitrary order on an unordered set. Citeseer's neighborhoods are small and interchangeable, so the extra expressiveness does not translate into accuracy. Max pooling answers "does any citing paper look like category c?", which is what this task needs.

Conclusion. For node classification on Citeseer, *max pooling wins*: vanilla-GCN accuracy at a fraction of LSTM's cost. LSTM is only worth the overhead on graphs with structured neighbor interactions that are small enough for the recurrence to fit in memory.